

# Security Audit Report

## PasswordStore Protocol

---

|                       |                                          |
|-----------------------|------------------------------------------|
| Auditor               | Carlos                                   |
| Date                  | April 2026                               |
| Commit                | 7d55682ddc4301a7b13ae9413095feffd9449b3f |
| Review Period         | 1 day                                    |
| Language              | Solidity 0.8.18                          |
| Lines of Code (nSLOC) | 20                                       |

### Findings

2 High · 0 Medium · 0 Low · 1 Informational

# Table of Contents

## 1. Executive Summary

## 2. Scope

## 3. Findings Summary

## 4. Detailed Findings

4.1 [H-01] On-chain storage of password defeats protocol purpose

4.2 [H-02] Missing access control on setPassword

4.3 [I-01] Incorrect NatSpec on getPassword

## 5. Disclaimer

# 1. Executive Summary

PasswordStore is a protocol designed to allow a single owner to store and retrieve a private password. The protocol consists of one contract with two core functions: setPassword and getPassword.

This audit identified **two critical vulnerabilities** that completely undermine the protocol's stated purpose. The first is a fundamental design flaw: storing the password on-chain means it is publicly readable by anyone, regardless of the Solidity private visibility modifier. The second is a missing access control check on setPassword, allowing any address to overwrite the owner's password.

Together, these findings render the protocol non-functional for its intended use case. The overall security posture is **critically insufficient** for deployment.

## 2. Scope

| File                  | nSLOC |
|-----------------------|-------|
| src/PasswordStore.sol | 20    |
| Total                 | 20    |

**Out of scope:** Tests, scripts, deployment configurations.

## 3. Findings Summary

| ID   | Title                                                 | Severity      |
|------|-------------------------------------------------------|---------------|
| H-01 | On-chain storage of password defeats protocol purpose | High          |
| H-02 | Missing access control on setPassword                 | High          |
| I-01 | Incorrect NatSpec on getPassword                      | Informational |

## 4. Detailed Findings

### [High] H-01: On-chain storage of password defeats protocol purpose

#### Summary

All data stored on-chain is publicly visible, regardless of the Solidity visibility modifier. The `s_password` variable is marked `private`, but this only prevents other contracts from reading it — anyone can read the storage slot directly.

#### Vulnerability Details

|          |                             |
|----------|-----------------------------|
| Severity | High                        |
| Type     | Data Exposure               |
| Contract | PasswordStore.sol           |
| Variable | s_password (storage slot 1) |

#### Root Cause

The protocol's entire value proposition is storing a password privately. However, the EVM stores all state variables in deterministic storage slots. The `private` keyword only restricts Solidity-level access from other contracts — it does not encrypt or hide the data. Any off-chain tool can read raw storage.

#### Impact

Any user can read the password at any time. The protocol provides zero actual privacy, giving the owner a false sense of security. This is a fundamental design failure — the protocol cannot fulfill its stated purpose.

#### Proof of Concept

1. Deploy PasswordStore and call `setPassword("mySecretPassword")`
2. Read storage slot 1 directly:

```
cast storage <CONTRACT_ADDR> 1 --rpc-url <RPC_URL>
```

3. Convert the returned bytes32 to a string:

```
cast parse-bytes32-string <RETURNED_VALUE>
```

4. The password is fully readable.

## **Recommended Mitigation**

Passwords should never be stored on-chain, even encrypted. The protocol's architecture needs a fundamental redesign. One approach: store a hash of the password on-chain and verify ownership off-chain. However, even a hash can be brute-forced if the password is weak. Overall, on-chain password storage is not a viable design pattern.

## [High] H-02: Missing access control on setPassword

### Summary

The setPassword function has no access control. Any external address can call it and overwrite the owner's password.

### Vulnerability Details

|          |                     |
|----------|---------------------|
| Severity | High                |
| Type     | Access Control      |
| Contract | PasswordStore.sol   |
| Function | setPassword(string) |

### Root Cause

The setPassword function is marked `external` with no modifier or `require` statement restricting it to the owner. The NatSpec states "This function allows only the owner to set a new password" but the code does not enforce this.

```
function setPassword(string memory newPassword) external {  
    // @audit – no access control!  
    s_password = newPassword;  
    emit SetNetPassword();  
}
```

### Impact

Any user can overwrite the stored password. This breaks the core invariant that only the owner can set the password. An attacker can set a new password, and the owner's next call to getPassword will return the attacker's value.

### Proof of Concept

1. Owner deploys and sets password to "ownerPassword"
2. Attacker calls setPassword("hacked") from a non-owner address
3. Transaction succeeds — no revert
4. Owner calls getPassword() and receives "hacked"

### Recommended Mitigation

Add an access control check at the top of setPassword:

```
if (msg.sender != s_owner) {  
    revert PasswordStore__NotOwner();  
}
```



## [Informational] I-01: Incorrect NatSpec on getPassword

### Summary

The NatSpec for getPassword documents a parameter newPassword that does not exist in the function signature.

### Details

```
/*  
 * @notice This allows only the owner to retrieve the password.  
 * @param newPassword The new password to set. // <-- wrong  
 */  
function getPassword() external view returns (string memory) {
```

The getPassword function takes no parameters. The @param tag referencing newPassword is a copy-paste error from setPassword.

### Recommended Mitigation

Remove the incorrect @param line from the NatSpec.

## 5. Disclaimer

This audit is a time-boxed security review and does not guarantee the absence of vulnerabilities. The findings reflect the state of the codebase at the specified commit hash. This report should not be considered an endorsement of the protocol. No audit can guarantee 100% security.

---

*Prepared by Carlos — Web3 Security Researcher*